

WHY PYTHON?



Easy to learn
and understand



Powerful and
widely used



Great for problem
solving



Used in real world
applications



A skill for your
bright future



PYTHON MADE SIMPLE

CODE • LEARN • SOLVE • SUCCEED



2026



A complete visual learning series for
Grade 10 students to build strong
foundations in Python programming.

WHAT YOU WILL LEARN



Basics &
Syntax



Operators &
Data Types



If Conditions



Loops



Lists, Tuples,
Dictionaries



And Much
More!



```
print("Hello, Python!")
```

```
if condition:
```

```
for i in range(5):  
    print(i)
```



★ Think. Code. Debug. Repeat. That's how great programmers are made! ★



ALGORITHM & FLOWCHART

THE BUILDING BLOCKS OF SOLVING PROBLEMS



WHAT IS AN ALGORITHM?

An algorithm is a **step-by-step set of instructions** to solve a problem or complete a task.



KEY POINTS

- Clear and unambiguous (no confusion)
- Steps are written in a specific order
- Must have a start and end
- Should produce the correct result
- Should be finite (ends after a limited number of steps)

EXAMPLE

Algorithm to make a cup of tea

- 1 Start
- 2 Boil water
- 3 Add tea leaves
- 4 Add sugar
- 5 Boil for 2 minutes
- 6 Pour in a cup
- 7 Serve the tea
- 8 Stop



REMEMBER

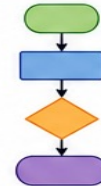


An algorithm is like a recipe – follow the steps in order to get the desired result!

WHAT IS A FLOWCHART?



A flowchart is a **diagrammatic representation** of an algorithm using standard symbols.



KEY POINTS

- Uses standard symbols and shapes
- Makes the logic easy to understand
- Shows the flow of control visually
- Helps in analyzing and communicating solutions better

COMMON FLOWCHART SYMBOLS

	Terminator	Indicates Start or End
	Process	An action or instruction
	Decision	A condition with Yes/No (True/False)
	Input / Output	Read input or display output
	Flow Line	Shows the direction of flow

EXAMPLE: Flowchart to make a cup of tea



WHY ARE THEY IMPORTANT?



Help in solving problems step by step



Improve logical thinking



Basis for programming and coding



Easy to communicate and share ideas



Reduce errors and save time



Think → Plan → Represent → Solve
That's the power of
Algorithm & Flowchart!





Python Programming

Simple • Powerful • Everywhere

Grade 10

1. INTRODUCTION TO PYTHON

Python is a high-level, interpreted, general-purpose programming language.

- ✓ Easy to learn and read
- ✓ Simple syntax similar to English
- ✓ Open-source and free
- ✓ Works on all major platforms (Windows, macOS, Linux)
- ✓ A large community with thousands of libraries



Python is beginner-friendly yet powerful enough for large projects!

2. ABOUT THE INVENTOR



Guido van Rossum

Year of Invention: **1991**

Python was created by Guido van Rossum and first released in February 1991.

Guido wanted a language that was easy to use, read, and maintain — and he named it “Python” inspired by the TV show “Monty Python’s Flying Circus”.

3. APPLICATIONS OF PYTHON

- Web Development**
Used in frameworks like Django, Flask.
- Data Analysis & Visualization**
Used with libraries like Pandas, NumPy, Matplotlib.
- Artificial Intelligence & Machine Learning**
Powering smart systems and predictions.
- Automation & Scripting**
Automate repetitive tasks and save time.
- Game Development**
Used in game engines like Pygame.
- Web Scraping**
Extract data from websites easily.
- Education**
Widely used for teaching programming.

4. WHY PYTHON FOR AI OR DATA SCIENCE?

- Simple & Easy**
Clean syntax makes it easy to learn and use.
- Great for Data Handling**
Excellent tools for collecting, cleaning, and analyzing data.
- Powerful Libraries**
Rich ecosystem like NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch.
- Perfect for AI/ML**
Built-in support for machine learning, deep learning, and neural networks.
- Large Community**
Huge support community and plenty of resources.
- Versatile & Scalable**
From small projects to large-scale enterprise solutions.

★ Python is the most popular language for AI and Data Science – and for good reason!

KEY FEATURES OF PYTHON

- High Productivity**
Do more in less code.
- Reliable & Safe**
Less error-prone and robust.
- Cross-Platform**
Run the same code anywhere.
- Extensible**
Add new features with libraries.
- Open Source**
Free to use and continually improved.

- 1. Source Code (HLL)**
Python code is written by the programmer.
- 2. Interpreter Reads**
Reads the code line by line.
- 3. Translate & Execute**
Each line is translated into machine language and executed immediately.
- 4. Output**
Result is shown to the user.

Example

```
>>> a = 5
>>> b = 3
>>> c = a + b
>>> print(c)
```

Output: 8

Used by:
 Python
IDLE, Jupyter
Other examples:
PHP, Ruby,
JavaScript (Node.js)

Works line by line. Good for development and debugging.

- 1. Source Code (HLL)**
Entire program is written by the programmer.
- 2. Compilation**
Compiler checks the whole code for errors and translates it into machine language.
- 3. Executable File**
Compiler creates an executable file (machine code).
- 4. Execution**
The executable file is run by the computer.
- 5. Output**
Result is shown to the user.

Compiles entire program at once. Faster execution.

Example
Source Code
program.py
↓
Compiler
↓
program.exe
(Machine Code)
↓
Run
↓
Output

Used by:
C, C++, C#,
Java, Go,
Rust, etc.

6. INTERPRETER vs COMPILER

Feature	INTERPRETER	COMPILER
Translation	Line by line	Entire program at once
Error Reporting	Stops at first error	Reports all errors after compilation
Executable File	Not created	Creates executable file
Speed	Slower (translation happens every time)	Faster (executes compiled code)
Debugging	Easy (line by line execution)	Harder (entire program compiled)
Memory Usage	Generally less	Generally more
Examples	Python, PHP, Ruby, JavaScript	C, C++, Java, C#, Go
Best For	Development, learning, scripting, testing	Large applications, performance-critical programs

DID YOU KNOW?

Python is consistently ranked as the top programming language in the world!



A SIMPLE PYTHON PROGRAM

```
1 # This program prints a message
2 print("Hello, World!")
3 print("Welcome to Python Programming!")
```

OUTPUT

```
Hello, World!
Welcome to Python
Programming!
```



Python: The language that turns *ideas* into *intelligent* solutions.





PYTHON & LANGUAGE TRANSLATION

From Human Language to Machine Language

Grade 10

1. PYTHON – A HIGH LEVEL LANGUAGE

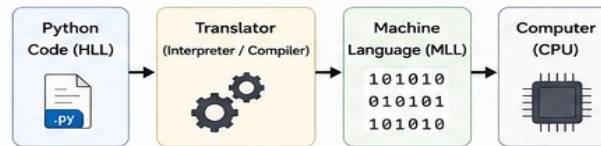
Python is a High Level Language (HLL). It is written in English-like statements that are easy for humans to read and understand.

```
print("Hello, World!")  
x = 10  
y = x + 5  
print(y)
```

- ✓ Closer to human language
- ✓ Easy to write and debug
- ✓ Independent of machine/CPU
- ✓ Needs conversion to run on a computer

2. HOW DOES CONVERSION HAPPEN? (HLL → MLL)

Computers understand only Machine Language (MLL) – binary code (0s and 1s). So, Python code must be converted before execution.



This translation process is done by two types of translators: Interpreter and Compiler.

3. ROLE OF INTERPRETER AND COMPILER

INTERPRETER

Reads the program line by line, translates each line into machine language and executes it immediately.

- No separate file created.
- Stops at the first error.

COMPILER

Reads the entire program at once, translates it into machine code, creates an executable file and then executes it.

- Creates a separate executable file.
- Shows all errors after compilation.



4. HOW INTERPRETER WORKS

- 1. Source Code (HLL)**
Python code is written by the programmer.
- 2. Interpreter Reads**
Reads the code line by line.
- 3. Translate & Execute**
Each line is translated into machine language and executed immediately.
- 4. Output**
Result is shown to the user.

Example

```
>>> a = 5  
>>> b = 3  
>>> c = a + b  
>>> print(c)
```

Output: 8

Used by:

Python
IDLE, Jupyter
Other examples:
PHP, Ruby,
JavaScript (Node.js)

Works line by line. Good for development and debugging.

5. HOW COMPILER WORKS

- 1. Source Code (HLL)**
Entire program is written by the programmer.
- 2. Compilation**
Compiler checks the whole code for errors and translates it into machine language.
- 3. Executable File**
Compiler creates an executable file (machine code).
- 4. Execution**
The executable file is run by the computer.
- 5. Output**
Result is shown to the user.

Example

```
Source Code  
program.py  
↓  
Compiler  
↓  
program.exe  
(Machine Code)  
Run  
↓  
Output
```

Used by:

C, C++, C#,
Java, Go,
Rust, etc.

Compiles entire program at once. Faster execution.

6. INTERPRETER vs COMPILER

Feature	INTERPRETER	COMPILER
Translation	Line by line	Entire program at once
Error Reporting	Stops at first error	Reports all errors after compilation
Executable File	Not created	Creates executable file
Speed	Slower (translation happens every time)	Faster (executes compiled code)
Debugging	Easy (line by line execution)	Harder (entire program compiled)
Memory Usage	Generally less	Generally more
Examples	Python, PHP, Ruby, JavaScript	C, C++, Java, C#, Go
Best For	Development, learning, scripting, testing	Large applications, performance-critical programs

KEY TAKEAWAY

Python is a high level language.
It uses an Interpreter to convert your code into machine language and execute it line by line.
This makes Python easy to use, flexible and great for beginners!



THE BIG PICTURE



You write
Python Code
(HLL)



Interpreter /
Compiler
Translates



Machine Language
(MLL – 0s and 1s)
101010
010101



Computer
Understands
and Executes



Output
You get
the result!



Understand the process. Write better code. **Build** the future!



JUPYTER NOTEBOOK

Your Interactive Workspace for Data Science



Grade 10

1. WHY JUPYTER NOTEBOOK FOR DATA SCIENCE?



Interactive & Easy to Use

Write code and see results immediately.



Visualize Data

Create charts and graphs right in the notebook.



Combine Code, Text & Visuals

Add explanations, maths, images and code in one place.



Great for Exploration

Perfect for experimenting and analyzing data.



Share Your Work

Share notebooks with others easily.



Supports Many Libraries

Works smoothly with NumPy, Pandas, Matplotlib, Scikit-learn and more.

2. HOW TO INSTALL JUPYTER NOTEBOOK

Option 1: Using Anaconda (Recommended for Beginners)

- 1 Go to: <https://www.anaconda.com/download>
- 2 Download Anaconda Installer (Windows / macOS / Linux)
- 3 Run the installer and follow the steps
- 4 After installation, open Anaconda Navigator
- 5 Click on "Jupyter Notebook" and launch

Option 2: Using pip (For Advanced Users)

- 1 Make sure Python is installed
- 2 Open Command Prompt / Terminal
- 3 Run the command:

```
pip install notebook
```

- 4 After installation, run:

```
jupyter notebook
```

Tip: Python 3.8 or higher is recommended.

3. HOW TO CONFIGURE FOR USE

- 1 Launch Jupyter Notebook (from Anaconda Navigator or by typing jupyter notebook in the terminal).
- 2 It opens in your web browser.
- 3 Default location is your Home directory.
- 4 To change the working folder:
 - Click "File" → "Open..."
 - Or use the "Upload" button to add files.
- 5 Create a new notebook: Click "New" → "Python 3"
- 6 You are ready to code!

★ All your notebooks are saved with .ipynb extension.

4. JUPYTER NOTEBOOK SCREEN – PART BY PART (FOUNDATION LEVEL)

Jupyter Logo
Takes you to the file browser (home page).

File Name
Shows the name of the current notebook (click to rename).

Menu Bar
Contains options like File, Edit, View, Insert, Cell, Kernel, Widgets, Help.

Toolbar
Quick access buttons for common actions like Save, Add Cell, Run, Stop, etc.

Kernel
Shows which Python kernel is running. (Usually Python 3)

Logout
Click to close the session and log out safely.

Jupyter My First Notebook Last Checkpoint: 2 minutes ago (unsaved changes) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 O

Cell Indicator
Shows the status of the cell.
In [] → Not run yet
In [1] → Run

Code Cell
Write and run Python code here. Press Shift + Enter to run.

Output Area
Displays the result of the executed code.

Markdown Cell
Write notes, explanations, titles, or text using Markdown. (Select Markdown from the dropdown and type.)

New Cell
Empty cell is ready for your next command.

```
In [1]: import pandas as pd
data = { 'Name': ['Ana', 'Ben', 'Cara'],
        'Score': [85, 90, 78]}
df = pd.DataFrame(data)
df
```

	Name	Score
0	Ana	85
1	Ben	90
2	Cara	78

```
In [ ]: """ This is a Markdown Cell
You can write **formatted text**, lists, images,
and more.
Markdown helps you explain your code and results.

In [ ]:
```

TYPES OF CELLS



Code Cell

Used to write and execute Python code. Performs calculations and shows results.



Markdown Cell

Used to write text, notes, headings, lists, images and explanations.

USEFUL SHORTCUTS

Shift + Enter	Run the cell and go to next cell
Ctrl + Enter	Run the cell (stay in the same cell)
A	Insert cell above
B	Insert cell below
D D	Delete selected cell
M	Change cell to Markdown
Y	Change cell to Code

WHY YOU WILL LOVE JUPYTER NOTEBOOK?

- ✓ Learn, code, visualize and document – all in one place!
- ✓ Perfect for Data Science, Machine Learning and AI projects.
- ✓ Helps you think, experiment and present your work clearly.



Jupyter Notebook – **Write. Run. Visualize. Explain. Repeat.**



Your journey to becoming a Data Science Expert starts here!



PYTHON OPERATORS & DATA TYPES

Know the Basics. Code with Confidence!

GRADE 10



1. VARIOUS OPERATORS (with Examples)

Type of Operator	Operator	Name	Example	Result
 Arithmetic Operators	+	Addition	7 + 3	10
	-	Subtraction	7 - 3	4
	*	Multiplication	7 * 3	21
	/	Division	7 / 3	2.333...
	//	Floor Division	7 // 3	2
	%	Modulus (Remainder)	7 % 3	1
 Comparison Operators	**	Exponentiation	7 ** 3	343
	==	Equal to	5 == 5	True
	!=	Not equal to	5 != 3	True
	>	Greater than	5 > 3	True
	<	Less than	5 < 3	False
	>=	Greater than or equal to	5 >= 5	True
 Comparison Operators	<=	Less than or equal to	5 <= 4	False
	and	Logical AND	(5 > 3) and (2 < 4)	True
	or	Logical OR	(5 < 3) or (2 < 4)	True
	not	Logical NOT	not (5 > 3)	False
 Assignment Operators	=	Assign	x = 10	x is 10
	+=	Add and assign	x += 5	x = x + 5
	-=	Subtract and assign	x -= 2	x = x - 2
	*=	Multiply and assign	x *= 3	x = x * 3
	/=	Divide and assign	x /= 2	x = x / 2
	%=	Modulus and assign	x %= 4	x = x % 4
 Identity Operators	**=	Exponent and assign	x **= 2	x = x ** 2
	//=	Floor divide and assign	x //= 3	x = x // 3
	is	True if both variables refer to same object	a is b	True / False
	is not	True if both variables refer to different objects	a is not b	True / False
 Membership Operators	in	True if value exists in a sequence	'a' in 'apple'	True
	not in	True if value does not exist in a sequence	'z' not in 'apple'	True



Operators are used to perform operations on values and variables.



2. DATA TYPES (with Examples)

Data types define the kind of value a variable can hold.

Data Type	Description	Example	Code Example	Output
 123 int (Integer)	Whole numbers (positive, negative or zero)	10, -5, 0, 1234	a = 10	10
 3.14 float (Floating Point)	Decimal numbers	3.14, -0.01, 2.5, 100.0	b = 3.14	3.14
 abc str (String)	Sequence of characters (text)	'Hello', 'Python', "Grade 10"	c = "Python"	Python
 T/F bool (Boolean)	Logical values	True, False	d = True	True
 list (List)	Ordered collection of items (mutable)	[1, 2, 3], ['a', 'b'], [10, 'hi', 2.5]	e = [1, 2, 3]	[1, 2, 3]
 tuple (Tuple)	Ordered collection of items (immutable)	(1, 2, 3), ('a', 'b')	f = (1, 2, 3)	(1, 2, 3)
 set (Set)	Unordered collection of unique items	{1, 2, 3}, {'a', 'b'}	g = {1, 2, 3}	{1, 2, 3}
 dict (Dictionary)	Collection of key-value pairs	{'name': 'Ram', 'age': 15}	h = {'name': 'Ram', 'age': 15}	{'name': 'Ram', 'age': 15}
 None NoneType	Represents no value or empty value	None	i = None	None



QUICK TIPS

- ✓ Use the right data type to save memory and avoid errors.
- ✓ Use type() function to check data type.

```
type(10)    # <class 'int'>
type(3.14)  # <class 'float'>
type("Hi")  # <class 'str'>
```

- ✓ Variables can store any data type.

```
x = 10
y = "Hello"
z = [1, 2, 3]
```



★ Practice makes perfect! Try writing small programs using different operators and data types.





PYTHON LOOPS

Loops help us repeat actions easily.
Write less code. Do more!



Grade 10

1. FOR LOOP – Repeat a block of code a fixed number of times.

What it does:

Iterates over a sequence (like list, string, range) or a fixed number of times.

Syntax:

```
for variable in sequence:  
    # code to repeat
```



Example 1: Print numbers 1 to 5

```
for i in range(1, 6):  
    print(i)
```

Output:

```
1  
2  
3  
4  
5
```

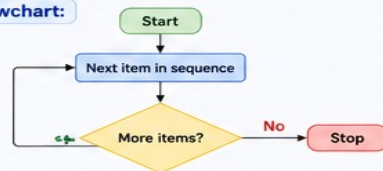
Example 2: Loop through a list

```
fruits = ["Apple", "Banana", "Mango"]  
for fruit in fruits:  
    print(fruit)
```

Output:

```
Apple  
Banana  
Mango
```

Flowchart:



Use for loop when you know how many times to repeat.

2. WHILE LOOP – Repeat a block of code while a condition is True.

What it does:

Repeats a block of code as long as the condition is True.

Syntax:

```
while condition:  
    # code to repeat
```



Example 1: Print numbers 1 to 5

```
i = 1  
while i <= 5:  
    print(i)  
    i = i + 1
```

Output:

```
1  
2  
3  
4  
5
```

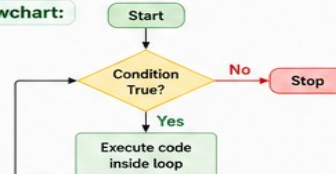
Example 2: Sum of first 5 natural numbers

```
i = 1  
sum = 0  
while i <= 5:  
    sum = sum + i  
    i = i + 1  
print("Sum =", sum)
```

Output:

```
Sum = 15
```

Flowchart:



Use while loop when you don't know how many times to repeat.

3. NESTED LOOP – A loop inside another loop.

What it does:

The inner loop runs completely for each iteration of the outer loop.

Syntax:

```
for i in sequence1:  
    for j in sequence2:  
        # code to repeat
```



Example: Print a 3 x 4 star pattern

```
for i in range(3):  
    for j in range(4):  
        print("*", end=" ")  
    print()
```

Output:

```
* * * *  
* * * *  
* * * *
```

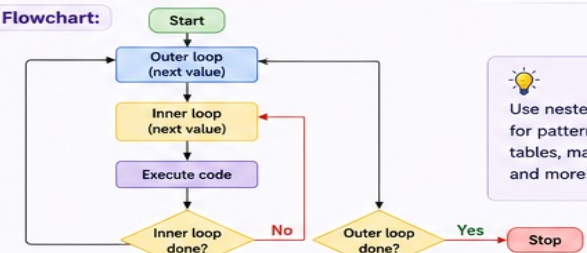
Example: Multiplication table of 5

```
for i in range(1, 11):  
    print("5 x", i, "=", 5*i)
```

Output:

```
5 x 1 = 5  
5 x 2 = 10  
...  
5 x 10 = 50
```

Flowchart:



Use nested loop for patterns, tables, matrices, and more!

FOR LOOP vs WHILE LOOP

Feature	for Loop	while Loop
Use when	Number of iterations is known	Number of iterations is unknown
Works with	Sequences (list, string, range, etc.)	Conditions
Structure	Simple and easy	Needs careful condition update
Risk	Less chance of infinite loop	Higher chance of infinite loop
Example	for i in range(5)	while i < 5

BREAK, CONTINUE & PASS

break

Stops the loop immediately.

```
for i in range(10):  
    if i == 5:  
        break
```

continue

Skips the current iteration and continues with next.

```
for i in range(5):  
    if i == 2:  
        continue
```

pass

Does nothing. Used as a placeholder.

```
for i in range(3):  
    pass
```

IMPORTANT TIPS

- ✓ Avoid infinite loops. Always update the condition in while loop.
- ✓ Use meaningful variable names.
- ✓ Test your code with different inputs.
- ✓ Indentation is important in Python!



Practice loops regularly to improve your logic and problem-solving skills!

★ Think. Plan. Code. Test. Loop your way to success!



Practice makes perfect!



Happy Coding! 😊



PYTHON IF CONDITIONS



If conditions help your program think and decide what to do next.



GRADE 10

Make Decisions in Your Code!

1. WHAT IS AN IF CONDITION?

An if condition is used to check a condition. If the condition is True, a block of code runs.

Basic Syntax:

```
if condition:  
    # code to run if condition is True
```



3. RELATIONAL OPERATORS (Used in Conditions)

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 3	True
<	Less than	2 < 4	True
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	3 <= 4	True

2. TYPES OF IF CONDITIONS (All)

1 IF STATEMENT

Executes code if the condition is True.

Syntax:

```
if condition:  
    # code
```

Example:

```
age = 16  
if age >= 18:  
    print("You are an adult.")
```

Output:

You are an adult.



2 IF-ELSE STATEMENT

Executes one block if the condition is True, and another if it is False.

Syntax:

```
if condition:  
    # code if True  
else:  
    # code if False
```

Example:

```
age = 16  
if age >= 18:  
    print("You are an adult.")  
else:  
    print("You are a minor.")
```

Output:

You are a minor.



3 IF-ELIF-ELSE STATEMENT

Checks multiple conditions one by one.

Syntax:

```
if condition1:  
    # code  
elif condition2:  
    # code  
else:  
    # code
```

Example:

```
marks = 85  
if marks >= 90:  
    print("A Grade")  
elif marks >= 75:  
    print("B Grade")  
else:  
    print("C Grade")
```

Output:

B Grade



4 NESTED IF STATEMENT

An if statement inside another if statement.

Example:

```
age = 20  
if age >= 18:  
    print("Eligible to vote")  
    if age >= 21:  
        print("You can also drive.")  
    else:  
        print("You cannot drive yet.")  
else:  
    print("Not eligible to vote.")
```

Output:

Eligible to vote
You cannot drive yet.



5 IF WITH MULTIPLE CONDITIONS

Uses and, or, not to combine conditions.

Examples:

a) Using and (both must be True)

```
x = 10  
y = 5  
if x > 5 and y < 10:  
    print("Both conditions True")
```

Output: Both conditions True

b) Using or (at least one True)

```
x = 10  
y = 15  
if x > 5 or y < 10:  
    print("At least one True")
```

Output: At least one True

c) Using not (inverts the condition)

```
x = 10  
if not x < 5:  
    print("x is not less than 5")
```

Output: x is not less than 5



4. TRUTH VALUES

Every condition evaluates to either:

True

(Yes, condition is satisfied)

False

(No, condition is not satisfied)

★ Python is case-sensitive; True and False must be written with capital T and F.

5. EXAMPLES IN REAL LIFE



Login Check

```
if username == "admin" and password == "1234":  
    print("Login successful")
```



Grade System

```
if marks >= 90:  
    grade = "A"  
elif marks >= 75:  
    grade = "B"  
else:  
    grade = "C"
```



Weather Check

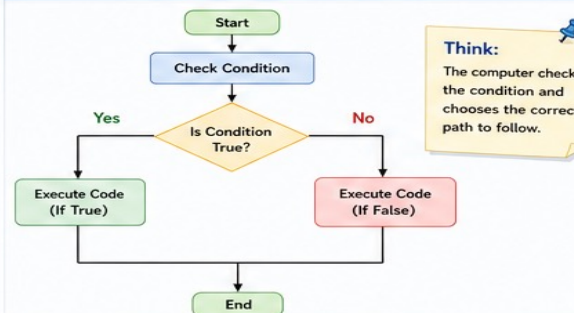
```
if temperature > 30:  
    print("It's hot today!")  
elif temperature >= 15:  
    print("Nice weather!")  
else:  
    print("It's cold today!")
```



Traffic Signal

```
if color == "Red":  
    print("Stop")  
elif color == "Yellow":  
    print("Get Ready")  
else:  
    print("Go")
```

6. FLOWCHART FOR IF-ELSE



Think:

The computer checks the condition and chooses the correct path to follow.

7. QUICK TIPS

- ✓ Indentation is important in Python.
- ✓ Use proper comparisons in conditions.
- ✓ Test your conditions with different values.
- ✓ Break complex conditions into smaller ones.
- ✓ Use meaningful variable names.

★ PRO TIP

Good use of if conditions makes your programs smart, interactive and useful!



Decide smartly. Code confidently. Think like a programmer!



Practice more, explore more, build more!



THANK YOU

